

System Architecture Document

Invoice Processing — From PDF to Decision | Practical MVP Architecture

1. Architecture Principles

- Multimodal-first document understanding for invoice PDFs/scans.
- Hybrid AI + deterministic controls: AI interprets; code validates and decides.
- Candidate retrieval before semantic reasoning; never expose the whole PO dataset to an LLM.
- Provider abstraction so Gemini/OpenAI/OpenRouter/Groq-backed model routes can be swapped.
- Auditability by default: every stage produces structured output.
- Simple enough to build and demo in one week.

2. Recommended Stack

Layer	Recommendation	Why
Frontend	Next.js + TypeScript + Tailwind	Fast implementation, clean dashboard, good demo
Backend	FastAPI + Python	Strong PDF/data/AI integration and rapid API development
Database	PostgreSQL	Relational fit for vendors, POs, invoices, runs, audit records
Object storage	S3-compatible storage; local disk for MVP	Preserve original PDFs and rendered pages
Validation	Pydantic + Python rule engine	Strict schemas and deterministic controls
Model access	Provider abstraction; Gemini/OpenAI primary, OpenRouter/Groq-compatible routes as options	Swap providers without changing workflow
Similarity	RapidFuzz + optional embeddings	Fuzzy identifiers plus semantic line matching
PDF rendering	PyMuPDF	Render PDF pages to images; lightweight
Deployment	Vercel frontend + Render/Railway/Fly.io backend + managed Postgres	Low-ops case-study deployment
Observability	Structured logs + run timeline; Sentry optional	Enough visibility for demo and debugging

3. Model Strategy

The supplied invoice benchmark supports native multimodal processing as the primary document strategy. For the MVP, the model provider is abstracted behind an interface such as `extract_invoice(document_images)` and `verify_invoice(extracted_json, document_images)`. A separate model call may verify uncertain fields, but the model cannot make the financial decision.

Task	AI?	Deterministic?	Notes
Read invoice layout/text	Yes	No	VLM
Extract fields/line items	Yes	Schema validation	Structured JSON
Resolve ambiguous descriptions	Yes	Yes	AI produces evidence; rules decide
Find PO candidates	Optional	Yes	DB retrieval first

Score candidate similarity	Optional	Yes	Weighted signals
Arithmetic	No	Yes	Recalculate independently
Tolerance	No	Yes	Policy engine
Duplicate detection	No/optional	Yes	Exact + fuzzy
Final decision	No	Yes	APPROVE/REVIEW/REJECT
Explanation wording	Yes	Yes	Template facts into readable explanation

4. High-Level Components

- Web UI: upload, live run timeline, invoice/PO comparison, decision, history.
- API Gateway/FastAPI: authentication, run orchestration, data access.
- Document Processor: PDF validation and page rendering.
- VLM Adapter: provider-independent extraction/verification.
- Extraction Normalizer: schema enforcement and normalization.
- Vendor Resolver: normalized exact/fuzzy matching.
- PO Candidate Service: indexed DB retrieval and filtering.
- PO Matcher: weighted multi-signal candidate scoring.
- Policy Engine: deterministic validation and decision.
- Duplicate Service: exact/hash/similarity checks.
- Audit/Run Service: stage events, evidence, reason codes.
- PostgreSQL: business/reference/run data.
- Object Storage: original PDFs and optional rendered images.

5. Data Flow

1. Browser uploads PDF to backend.
2. Backend stores the original file and creates RUNNING run.
3. Document Processor renders pages.
4. VLM Adapter extracts structured invoice JSON.
5. Normalizer validates schema and calculates extraction status.
6. Vendor Resolver maps invoice vendor to vendor master.
7. PO Candidate Service retrieves candidates using explicit PO reference if available; otherwise vendor/currency/amount/date/item signals.
8. PO Matcher scores candidates and records alternatives.
9. Policy Engine validates the top candidate and remaining PO balance.
10. Duplicate Service checks historical invoices.
11. Decision Engine combines blocking rules, confidence, match margin and duplicate status.
12. Audit Service stores all stage outputs and emits events for the live UI.
13. Frontend displays decision and evidence.

6. Suggested API Endpoints

Method	Endpoint	Purpose
POST	/api/runs	Create invoice-processing run/upload
GET	/api/runs/{run_id}	Get run status and result
GET	/api/runs/{run_id}/events	Stream/poll stage events
GET	/api/runs/{run_id}/evidence	Get extraction/match/validation evidence
GET	/api/pos?query=	Search PO candidates
GET	/api/vendors	List/search vendors
POST	/api/runs/{run_id}/review	Resolve/override review
GET	/api/runs	History/filtering

GET	/api/health	Health check
-----	-------------	--------------

7. Database Model

Core tables: users, vendors, purchase_orders, purchase_order_lines, invoices, invoice_lines, processing_runs, run_events, po_candidates, validation_results, duplicate_matches, decisions, decision_audits.

8. PO Matching Implementation

For MVP, use PostgreSQL indexes plus normalized text and RapidFuzz. If the PO set grows, add pgvector for embeddings. Candidate generation should be deterministic and narrow; semantic matching is used only for descriptions/references that are difficult to normalize.

9. Security Architecture

- TLS between browser and backend.
- Server-side model credentials.
- Role-based API authorization.
- Signed/private object-storage URLs for documents.
- Parameterized SQL/ORM queries.
- Upload size/type validation.
- No raw API keys, invoice contents, or secrets in frontend logs.
- Production deployment should define retention, encryption at rest, provider data-retention policy, and audit access.

10. Deployment

14. Frontend deployed as a Next.js application.
15. FastAPI deployed as one backend service with environment-based secrets.
16. PostgreSQL managed service.
17. Object storage bucket for invoice PDFs.
18. Optional Redis only if asynchronous jobs become necessary; omit for MVP.
19. CI pipeline runs lint, unit tests, API tests and builds before deployment.

11. Scalability Path

- Move processing into a queue/worker model for hundreds/thousands of invoices.
- Add Redis/Celery or a managed job queue only when asynchronous load requires it.
- Add pgvector for large PO semantic retrieval.
- Cache vendor/PO reference data.
- Batch model calls only when safe; never batch across financial decisions if it obscures auditability.
- Add provider fallback only for extraction failure; never silently change decision policy.

12. Architecture Trade-offs

Choice	Decision	Reason
VLM vs OCR-first	VLM-first	Benchmark supplied shows stronger native image performance on noisy invoices/receipts
LLM-only PO matching vs hybrid	Hybrid	Financial decisions need deterministic controls and auditability

Microservices vs monolith	Modular monolith	One-week build; clear internal modules without deployment overhead
Vector DB vs PostgreSQL	Postgres first	Small seeded PO dataset; add pgvector only if needed
Real ERP integration vs fixture data	Fixture data	Case study explicitly allows realistic test data and avoids unnecessary scope